

Security Analysis of Dependency Confusion for Third Party Components: A Risk Assessment Framework Based on Source Code Tree Matching

Yanwen Ding

Beijing University of Posts and Telecommunications

Weihan Tian

Beijing University of Posts and Telecommunications

Baojiang Cui

cuibj@bupt.edu.cn

Beijing University of Posts and Telecommunications

Article

Keywords: software security, component security; dependency confusion; code auditing, software supply chain security

Posted Date: October 18th, 2024

DOI: <https://doi.org/10.21203/rs.3.rs-5261746/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Security Analysis of Dependency Confusion for Third Party Components: A Risk Assessment Framework Based on Source Code Tree Matching

Yanwen Ding(丁彦文), Weihai Tian(田维翰) Baojiang Cui*(崔宝江)

Beijing University of Posts and Telecommunications, Haidian District, Beijing, China

Abstracts

With the widespread use of third-party components in software development, their security has become a major challenge in the field of software security. The implantation of malicious components or the unintentional reference of security weak components by developers may lead to serious security problems. In order to effectively solve the dependency obfuscation problem, this paper proposes a detection framework (Comchecker) based on source code analysis to cope with the dependency obfuscation problem through source code analysis techniques. The framework first accurately identifies and builds the code tree of third-party components and optimizes them through static analysis of component source code. Then, it evaluates the security risk of the components by designing a comprehensive evaluation model which combines various factors such as the source, structural features, and other components. Subsequently, we conduct a series of experiments to evaluate the proposed framework and compare it with state-of-the-art component detection tools. Through extensive experiments with three comparative detection tools, our detection framework achieves 93.51% accuracy for third-party component detection, outperforming existing detection tools in terms of detection accuracy and precision, and 91.04% accuracy for obfuscation-dependent detection. Finally, Comchecker is also well extended to be applied to components used in most mainstream development languages.

Keywords

software security; component security; dependency confusion; code auditing; software supply chain security

1.Introduction

With the growing maturity of the software development ecosystem, the use of third-party components has become an important means of accelerating the software development process. It can provide off-the-shelf solutions that drastically reduce the time and cost of development. However, the extensive use of third-party components also brings new challenges, especially in the area of software security [1]. In the latest OWASP TOP10 [2], Vulnerable and Outdated Components have risen up the rankings and have come to the sixth position and is the only category that has no vulnerability and common exposure (CVEs) mapped to the included CWEs. Security issues in the components used, such as malicious code injection, vulnerability exploits [3], etc., may affect the security of the entire software system, causing data leakage, remote execution, etc. 3.97 billion vulnerable components are used in an average of 37.8 billion downloads per month on Maven Central [1]. Some popular components are used on hundreds of thousands of websites, and an attacker using dangerous code in one of these components could expose hundreds of thousands of sites to attacks, triggering a serious security incident. In 2017, a backdoor version of CCleaner, a system maintenance tool for Windows, was downloaded more than 2.3 million times in one month on the vendor's official website after being implanted with malicious code [4]. Alex Birsan [5] exploited dependency confusion to successfully compromise several major tech companies such as Apple and Netflix by uploading malicious Node packages. In 2024, attackers used auto-typography to publish more than five hundred packages to PyPI in a matter of hours [6], with the names of the packages intentionally designed to be misspellings of popular libraries such as "pillow@1.0.0" as "pilloa@1.0.0", and developers who do not look carefully at the component information are likely to accidentally install these malicious components instead of the ones they originally intended to use.

Dependency confusion is a heavily present security issue that arises during the use of third-party components, and attackers utilize dependency confusion attacks when they trick package managers into extracting malicious components from public repositories instead of legitimate ones by disguising them as legitimate components [7, 8, 9, 10]. It exploits the developer's negligence in specifying the source repository, which in turn automatically handles the download and configuration of the malicious code through the component installer.

Common dependency confusion mainly includes npm confusion, PyPI component confusion, JAR dependency confusion, etc. Dependency confusion vulnerabilities are very common in Java applications. During the build process, developers often package project dependencies into JAR files and deploy them to the production environment. However, the lack of comprehensive analysis and validation mechanisms for dependencies in build tools such as Maven and Gradle, as well as possible negligence on the part of developers when configuring dependency information, leads to an increased risk of dependency confusion attacks. For example, a developer may misconfigure the version of a dependency [11] or omit necessary dependencies, allowing an attacker to exploit these flaws and inject malicious code

into the application. In addition, due to the presence of a large number of third-party libraries in the Java ecosystem, these libraries themselves may have security vulnerabilities that, once exploited by an attacker, can also lead to dependency confusion attacks. Dependency confusion vulnerabilities in the Python language can similarly pose a threat to the security of an application. In the Python build process, the requirements.txt file is typically used to record project dependencies and install them using the pip tool. However, the requirements.txt file lacks strict constraints on dependencies, and it is unable to recognize the version range of a dependency or ensure that the source of the dependency is safe and secure. This allows an attacker to exploit vulnerabilities in the requirements.txt file to inject malicious code into an application [7, 12, 9]. For example, an attacker can publish a seemingly normal library in a public code base, but it actually contains malicious code. When a developer adds this library to the requirements.txt file and installs it using the pip tool, malicious code can be injected into the application, leading to security problems. Additionally, some developers may install dependencies directly using the pip install -r requirements.txt command and ignore the --no-deps option, in which case the pip utility automatically parses and installs all the dependencies, further increasing the risk of vulnerability in the application. Dependency confusion vulnerabilities are often found in JavaScript applications. package.json files embed project-wide dependencies into public scripts as part of the build process. Since npm is the main repository for Node.js packages, it is susceptible to dependency confusion attacks during the development process if there is a situation where the developer fails to include proper configuration information in the package manager or if there is a lack of the correct internal registry[10]. In related research, Zhao[13] use deep neural networks to predict whether a program is obfuscated or not by using semantic information from disassembled binaries. Snyk provides snyk[14] to detect dependency confusion in npm by using whether the package name is registered in the public registry. Sonatype[15] provides namespace confusion checking script to check whether the artifacts between the user's repository contains artifacts with the same name, identifying possible dependency confusion attacks. Orca[16] utilizes the SideScanning technique to detect and parse dependency definitions in common files (e.g., requirements.txt for Python and package.json for JavaScript) and extract a list of dependencies. This list is then compared to the public registries of PyPI and npm and alerts are issued for any packages that are not registered there. Jukka Ruohonen [17] examined the CVSS (Common Vulnerability Scoring System) scores of vulnerable Python packages as well as the corresponding weaknesses based on CWE (Common Weakness Enumeration).

Phylum's Q1 2024 Evolution of Software Supply Chain Security Report [22] analyzed 186 million files in 2.9 million packages, of which 24129 packages executed suspicious code during installation, 9650 packages attempted to obfuscate the underlying code, 428706 packages contained precompiled binaries, and 1369 packages secretly downloaded and executed code from remote sources. In March 2024, Phylum also detected a large-scale mis-planting campaign, where a large number of fake variants of packages, such as colorama, requests, etc., were created for PyPI. This operation can trick developers into mistakenly downloading malware packages, which allows attackers to steal passwords, cookies and even compromise systems. This highlights the fact that threats in the software supply chain are persistent and serious, and the rapid proliferation of threats can have a significant impact before they can be detected and remediated.

The approach we present in this paper has three main contributions:

1. Comchecker: A source code-based dependency checker for Python, Java, two languages that make heavy use of third-party component packages, to quickly locate components or other risks of dependency confusion.
2. Our tool taps into regular SBOM analysis, enhancing its ability to accurately analyze component dependencies and locate risks even without relying on configuration files.
3. We enhance the recognition-dependent comparison algorithm to be able to drastically reduce the number of comparisons, improve detection efficiency, and reduce resource usage without decreasing the accuracy.

The rest of the paper is organized as follows. In Section 2, we focus on the background and motivation of our study. In Section 3, we describe the detection method of our tool in detail. In Section 4, we present our data set, experimental details, and evaluation metrics. In Section 5, we evaluate the detection results of our tool against the mainstream tools of the day and provide conclusions. We discuss the limitations in Section 6. Finally, in Section 7 we plan future work and conclude the paper.

2. Background and motivation

SCA. SCA stands for Software Component Analysis [18] and is an automated process for identifying open source software in code. SCA mainly examines package managers, source code, container images, manifest files, etc. Identified open source components, license compliance data, etc. generate a bill of materials (BOM) according to certain rules, and then compare them with various databases, including NVD, Snyk, etc. A good SCA requires a comprehensive database, extensive language support, risk prioritization and remediation and is developer friendly, among other requirements [19].

SBOM: Software Bill of Materials (SBOM) is a detailed list of software components and third-party components used in the deployment process and their versions. The main formats of SBOM are SPDX, CycloneDX, and CPE, etc. The main functions are to increase the transparency of the software supply chain, document component compliance, and perform risk assessment and security management. SBOM is an essential and important part of the software development and maintenance process [20], which can better help organizations and development teams to manage and take control of software security and compliance.

Many solutions exist for dependency confusion, such as reviewing the sources, maintainers, and community ratings of dependencies before introducing new dependencies [7, 8, 21], installing packages using their fully-qualified names and exact version numbers, and also using tools such as Sonatype, Orca[23], etc., to identify which components are used in the software and comparing them with the public registry, and alerting the unregistered packages to alert them.

The above solutions are more from the perspective of prevention and through the review of configuration files or SBOM to solve the dependency confusion, but for projects that are already packaged or running, which may contain a large number of pre-compiled bytecode files, or in the configuration file does not contain a complete component dependency information, at this time, if there is a latent malware packages, it will be difficult to be detected by conventional means. It is almost impractical for developers to manually audit the

source code of these packages, which contain a large number of pre-compiled binary files. Research [1] has shown that code reviews and binaries are the most effective way to identify vulnerability risks, and that developers download an average of about 24 billion packages per week from the JavaScript package management tool npm, but almost no one verifies the integrity of the downloaded code. Not only does manual verification fail to guarantee the accuracy of the verification, it also takes many time and affects the development process (In our experiments, it takes on average 0.5h to manually verify the security of a package). Therefore having automated tools is the key to solving the problem of quickly locating dependency confusion.

3.Methodologies

Our tool can perform component identification in three ways, analysis of SBOM (3-A), analysis of configuration files (3-B), and analysis of source code (3-C). The main flow of tool analysis is shown in Fig. 1, where detailed information about the component is finally obtained by analyzing it in three ways. In the following we will describe the specific methods of our analysis.

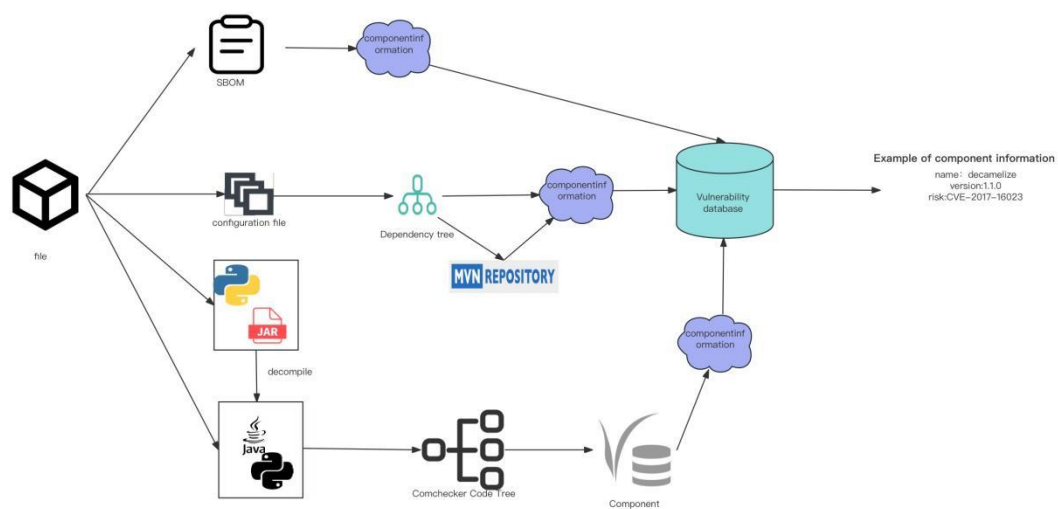


Fig. 1 Comchecker analysis flow.

A.SBOM analysis

The analysis of the SBOM means that the software has provided a complete SBOM and we use Comchecker to analyze the SBOM directly. The configuration of a component is identified by the CPE [24] field in the SBOM, CPE is an entity that uses the URL format to represent an IT product, which contains many attributes, in the example in Figure 2, 2.3 denotes the version number of the URL specification, a denotes an application program, and other possible values are h (hardware) or o (operating system), decamelize_project denotes the the organization that maintains the product, decamelize denotes the product name, and 1.1.0 is the version number of the product. We compared the product name and version

number obtained by CPE with our database, and finally detected an Improper Input Validation risk for the decamelize component used in the example SBOM, with the number CVE-2017-16023 [25].

```

▶ bom-ref:      "pkg:npm/decamelize@1.1.0...age-id=76f3451d989a243a"
  type:         "library"
  name:         "decamelize"
  version:      "1.1.0"
▼ cpe:          "cpe:2.3:a:decamelize_project:decamelize:1.1.0:*:*:*:*:*:*"
  purl:         "pkg:npm/decamelize@1.1.0"
▼ properties:
  ▼ 0:
    name:       "syft:package:foundBy"
    value:      "javascript-lock-cataloger"
  ▼ 1:
    name:       "syft:package:language"
    value:      "javascript"

```

Fig. 2 SBOM example.

B. Configuration file analysis

Of course, most software does not come with a complete SBOM, but for software that uses a package management tool to manage dependencies, we can also analyze the configuration file to get component information and then conduct risk analysis. We take Maven as an example, after locating the parent module and submodules, analyze the groupId, artifactId and version fields in the dependencies of each module to get the name and version number of the component and inherit the dependency management [26] of the parent module, and recursively create a dependency tree to analyze each dependency. By comparing the dependency information of each node in the dependency tree with the central repository, we can improve the information of the dependency tree and finally obtain a complete dependency tree. Then we compare it with the vulnerability database to get the components that are at risk.

C. Source code analysis

Figure 3 outlines our analysis methodology based on source code analysis, where a structured approach was used to perform the analysis of the Python code. First, by traversing the root directory, we analyzed the specific structure of the code files and determined their types. During the resource location process, we prioritized the use of source code for analysis if the code was provided in source code form, because analyzing the source code directly allows us to complete the evaluation of components more efficiently. If a .pyc bytecode file is encountered, we will decompile it using the uncompyle6 [27] tool and use the resulting source code as input for subsequent analysis.

Subsequently, we used Python's ast module to parse the source code into an abstract

syntax tree (AST) [28]. During the parsing process, we extracted key nodes in the code such as function calls, class definitions, module imports, function definitions, etc., and constructed the corresponding code tree structure. Then, we optimize the generated code tree to improve its readability and analysis efficiency. In order to assess the trustworthiness and potential risks of the code, we obtained the source code of the component from PyPI and generated the corresponding code tree following the same AST parsing process. After that, we compared the code tree to be analyzed with the code tree of the PyPI component in detail, and through the comparison results, we derived the component information and identified the possible security risks. This process realizes an accurate mapping between Python code files and PyPI packages, which provides strong support for code quality assessment, security auditing and compliance checking.

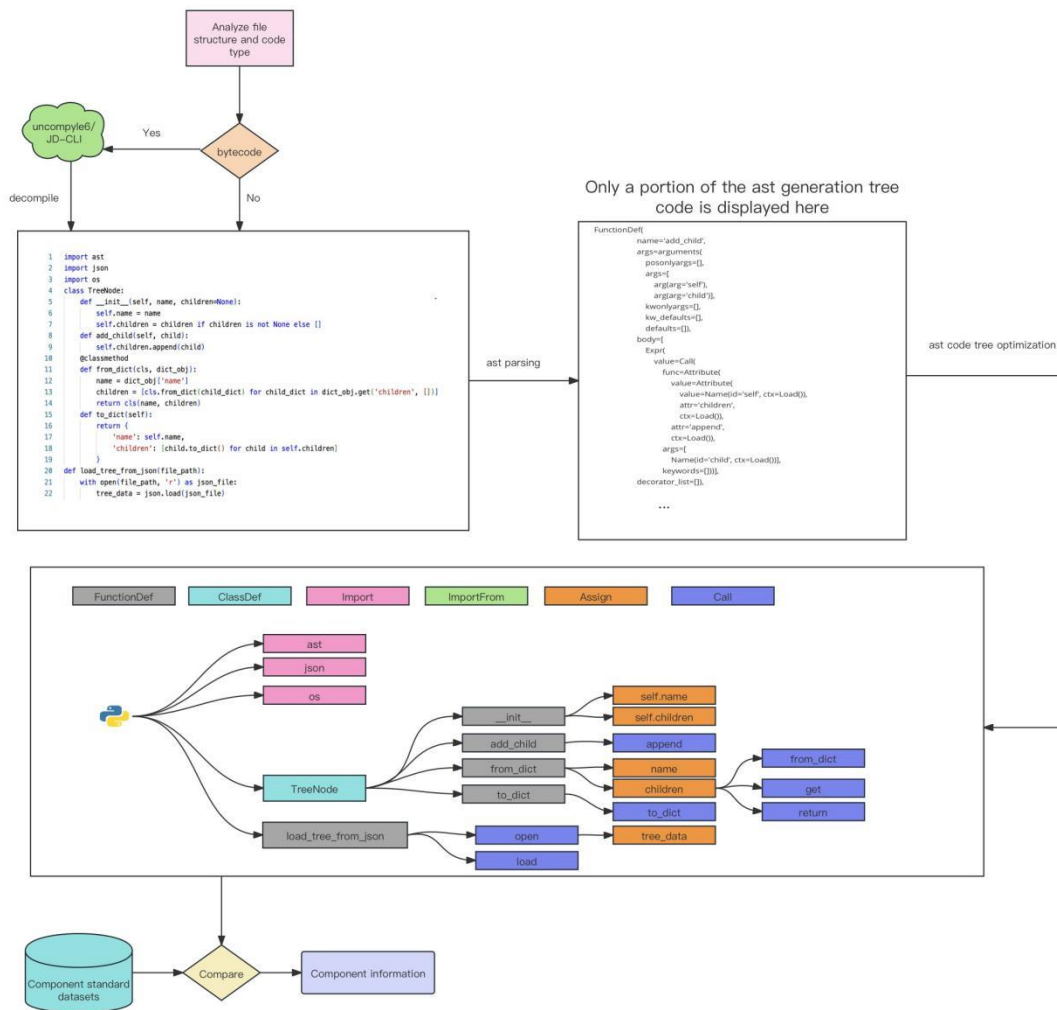


Fig. 3 Analyzing components based on source code.

Python's ast parsed nodes have a variety of type definitions [28], such as FunctionDef, Import, Assert, ClassDef, Call, Expr, Nameconstant, and so on. We use AST for source code parsing, the generated code tree is complete, 123 lines of source code in our sample program, the spanning tree of up to 5377 lines of code, although this is conducive to the understanding of the code hierarchy, but for the detection of component information, do not need such a

complex spanning tree to determine the specific information of the component, large amounts of redundant information can take up many computational resources when comparing with existing data, and for large projects, this can significantly increase the load on the database. For these reasons, we optimize the spanning tree to retain as much information as possible that is useful for identifying components. Our optimization retains the node types FunctionDef, ClassDef, Import, ImportFrom, Call, and Assign as the optimized code tree node types, and removes the node types Expr, Str, Num, NameConstant, and Pass, while we chose the node types Try, While, and For as alternatives. When the component information cannot be determined by the existing node types, the component identification accuracy is increased by the comparison of alternative nodes.

The comparison process is shown in Fig. 4, where we take a structured and hierarchical approach. First, the code tree to be analyzed was subjected to an initial structural analysis to determine its number of layers and the number of child nodes per layer. Then, we use these structural features to filter the component code trees in the component standard database and retain only those trees whose number of layers and number of child nodes per layer match the target tree as alternative trees for further comparison.

During the comparison of alternative trees, we follow a breadth-first traversal strategy starting from the root node. First, the node information of the first layer of child nodes is compared. If all the first level child nodes are the same, the comparison continues recursively to the next level until the end of the tree or different nodes are found. As soon as a mismatched node is found during the comparison process, we immediately stop the comparison of the current alternative tree and start the comparison of the next alternative tree instead.

Through this layer-by-layer comparison, we can effectively reduce the number of unnecessary inter-tree comparisons, thus significantly improving the search efficiency. Eventually, when we successfully find a component code tree in the database that is identical to the target tree structure and node information, it is considered that we have found the same component code tree. If no match is found after traversing all the alternative trees, it means that there is no component code tree in the database that is exactly the same as the target tree.

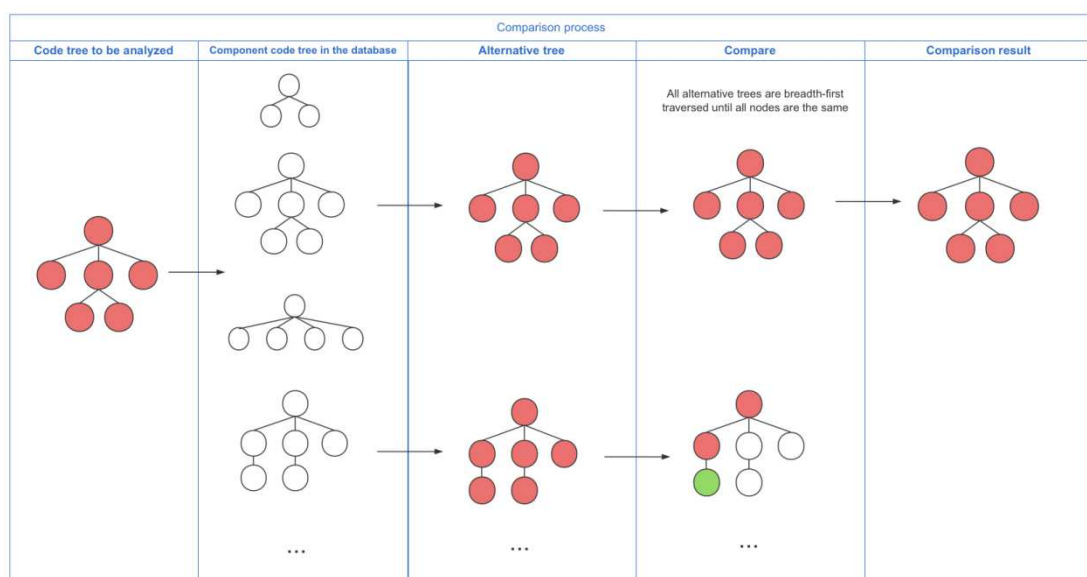


Fig. 4 Comparison process of code tree.

4.Experimental evaluation

In this section, we conduct a number of experiments to demonstrate the effectiveness and superiority of our proposed approach, and we compare Comchecker with the current mainstream SCA tools on the test data set.

A.data set

Our experiments use the test data set for comparison experiments. Of course, prior to the experiment we had already constructed the standard database of components in our database for making comparisons.

1) Component Standards Database

This data set is created by customizing a crawler that crawls the source files of different versions of different components from the Maven central repository [29] and PyPI official website [30], and then preprocesses them to generate a code tree in the same form as the code of the files to be compared. Because the number of components and versions is too large, we get 100000 component information and source files from each of the two repositories as our component standard database.

2) Test Data Set

The test data set is adapted from existing software, the project used 255 jar packages, 102 PyPI packages. In order to verify the tool's detection capability, we do not write all the above components into the configuration file, 17 jar packages dependencies were introduced in pom.xml, and 82 components are declared in requirements.txt.

3) Confusion Test Data Set

The confusion test data set consists of 44 jar packages and 50 PyPI packages randomly selected from the test data set and written to pom.xml and requirements.txt respectively. From these, 14 jar packages and 15 PyPI packages were selected to add obfuscated code, and another 10 jar packages and 18 PyPI packages were selected to obfuscate their component names or version numbers.

B.Experimental details

We propose Comchecker for recognizing component information. Compared to existing component identification tools [19], our proposed Comchecker utilizes source code analysis to identify component information more accurately and obtain information that cannot be captured by other tools.

We compare our framework to existing state-of-the-art detection tools, and we compare it to three tools that are currently popular on GitHub: Syft [31], Dependency-check [32], and OpenSCA [33]. All of these tools support the detection of open source dependencies, licenses, etc.

In the experimental setup, in order to comprehensively test the performance of the tools, we conducted a large number of comparative experiments on the four tools in the same environment, through the experiments of identifying component information on the test data set and identifying the components after confusion on the confusion test data set, to validate the performance of the different tools and to explore the advantages of Comchecker. Component information includes component name and version number, and confusion refers to artificial changes to the code of dependent components, including variable renaming, function renaming, code structure change, code content deletion, and similar name naming, in order to simulate the dependency confusion that may be encountered in the actual development process.

C. Assessment of indicators

We selected accuracy, precision, detection efficiency, false-negative rate, and F1-Score [34] to evaluate the tool. Let TP_1 denote the number of used components identified by the tool, FP_1 denote the number of components identified by the tool but not actually used, FN_1 denote the number of components used but not detected by the tool, TP_2 denote the number of components with dependency confounding correctly identified by the tool, FP_2 denote the number of components with dependency confounding incorrectly identified by the tool, TN_2 denote the number of components with dependency confounding that are not correctly identified by the tool. FN_2 denotes the number of components with dependency confusion missed by the tool.

CFC denotes the number of obfuscated code components successfully identified by the tool, and CFI denotes the number of obfuscated message components successfully identified by the tool. $Accuracy_1$ is used to measure the proportion of identified components among all used components, and is computed using the formula:

$$Accuracy_1 = \frac{TP_1}{TP_1 + FP_1 + FN_1} \quad (1)$$

$Accuracy_2$ measures the percentage of correctly detected dependency confusion components out of all components with dependency confusion, calculated as:

$$Accuracy_2 = \frac{TP_2}{TP_2 + FP_2 + FN_2 + TN_2} \quad (2)$$

$Precision_1$ is the ratio of identified used components to all identified components, calculated as:

$$Precision_1 = \frac{TP_1}{TP_1 + FP_1} \quad (3)$$

$Precision_2$ is the proportion of components detected as having dependency confusion that actually have dependency confusion, calculated as:

$$\text{Precision}_2 = \frac{TP_2}{TP_2 + FP_2} \quad (4)$$

Detection Efficiency We use the time required to identify a component as a quantitative criterion, and FNR_1 is the ratio of components that are not detected to all used components, calculated as:

$$FNR_1 = \frac{FN_1}{TP_1 + FN_1} \quad (5)$$

FNR_2 is the ratio of the number of components with dependency confusion that were missed to the number of components that actually have dependency confusion. The calculation formula is:

$$FNR_2 = \frac{FN_2}{TP_2 + FN_2} \quad (6)$$

The F1-Score is the reconciled mean of precision and recall and is used to measure the overall performance of the model. The calculation formula is:

$$Recall = \frac{TP}{TP + FN} \quad (7)$$

$$F1 - Score = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (8)$$

5.Results

In this section we will show the results of four tools in the test data set experiments. In order to increase the reliability of the findings, we also provide the test data set and confusion test data set used as well as the results of the assays [35].The following are the Java and Python testing results respectively.

Table 1

Java dependency information identification results.

Tool	Syft	OpenSCA	Dependency-check	Comchecker
Accuracy ₁	0.0675	0.9608	0.0675	0.9961
Precision ₁	0.7391	0.9800	0.7727	1.0000
Detection efficiency (s)	0.3250	3.0241	5.6121	4.4940
FNR_1	0.9318	0.0196	0.9318	0.0039
$F1\text{-Score}_1$	0.1384	0.9800	0.1481	0.9961

From Table 1, we can see the detection results of Java dependency. The accuracy rates of Syft, OpenSCA, and Dependency-check are 6.75%, 96.08%, and 6.75% respectively, and the accuracy rate of Comchecker is 99.61% , Syft and Dependency-check can only analyze the jar

packages in the configuration file, and Maven, for example, determines the dependency information by analyzing the pom.xml file, so the two tools can only identify 17 direct dependencies declared in the configuration file in the test data set, and the remaining 238 indirect dependencies can not be identified. OpenSCA, on the other hand, can obtain indirect dependencies on the basis of the configuration file by looking up the Compile Dependencies information in the Maven central repository. However, in our tests, although OpenSCA can get indirect dependencies by querying the central repository, it can't judge the dependencies with <optional> elements and chooses not to detect them, and the identification of direct dependencies is not complete enough. OpenSCA is the tool with the best detection effect except for our tool, which identifies all direct dependencies, and most of indirect dependencies. It recognizes all direct dependencies, and most indirect dependencies. Comchecker can analyze the dependencies at the code level based on the configuration file and compare the result with the information in the central repository, so we not only identified all direct dependencies, but also most of the indirect dependencies, including the ones that are not declared in the pom.xml. There is one indirect dependency that we did not successfully identify because of the following two reasons: 1) the source code of the indirect dependency is too short and the generated code tree is not enough to locate the exact dependency information. 2) Since we have to decompile the jar file first and then generate the code tree, the difference between the decompiled result and the real source code also affects our identification result.

The precision of the four tools' detection is 73.91%, 98.00%, 77.27%, and 100.00%, and the presence of non-component results in Syft's detection results affects the accuracy of its detection. For component version number detection, the identification function of Syft, OpenSCA, and Dependency-check relies on the dependency version declaration of the configuration file, but in addition to the version number of the stable version declared in the configuration file, the following identification error cases occur: 1) The version number is SNAPSHOT to indicate a temporary version under development. 2) In the event of a dependency conflict, the system does not use the version declared directly in the configuration file according to the 'shortest path first' principle. 3) When using automated tools such as Maven Versions Plugin, the dependency version is automatically updated but the configuration file is not yet updated. The above situations lead to a large gap between the three tools' identification of dependency version numbers and the actual dependency application. Comchecker confirms the version numbers not only by analyzing the configuration files, but also by analyzing the source code. One of the most important factors affecting the identification results is the number of components in the database that have the exact version. In our experiments, Comchecker accurately detects dependency version numbers in the test data set, and successfully detects dependency conflicts and lagging configuration file updates caused by automated tools.

Regarding the detection efficiency of the four tools, Syft takes the shortest time because it only needs to identify and detect configuration files, OpenSCA takes longer because it needs to check the data in the central repository and match the risk information, and Dependency-check takes longer because it needs to check and update the NVD database each time it checks and match the risk information. The average detection time of our tool is 4.494s because it involves source code detection and needs to compare the generated code tree with

the database, but we think this time is worthwhile in light of the more dependency information we have detected.

Table 2

Java dependency confusion identification results

Tool	Syft	OpenSCA	Dependency-check	Comchecker
CFC	0	0	0	13
CFI	0	0	10	10
Accuracy ₂	/	/	0.7692	0.9643
Precision ₂	/	/	1.0000	0.9565
Detection efficiency (s)	/	/	5.6121	4.4940
FNR ₂	/	/	0.5833	0.0435
F1-Score ₂	/	/	0.6667	0.9565

Syft, OpenSCA does not have the ability to detect dependency confusion because it does not check bytecode or source code, nor does it check the contents of configuration files. For the dependencies where we add obfuscated code and obfuscated information, the above tools can neither check the component name or the component version number, nor analyze the code within the component, as shown in Table 2. Dependency-check can search the artifactId, url, groupId, version and other fields in the jar package, and can recognize the confusion of component name or version number, but cannot recognize the inserted obfuscated code. Comchecker identified 10 dependency confusions and 13 out of 14 code dependency confusions, one case of code dependency confusion was not successfully identified and one case of code confusion was identified as a code-obfuscated component without obfuscated code. The reason we analyzed was that the obfuscated code we added was too short or the different versions of the component code did not differ in our analysis granularity, which did not affect the structure of the code tree, and therefore were not identified correctly.

Table 3

Python package detection results

Tool	Syft	OpenSCA	Dependency-check	Comchecker
Accuracy ₁	0.5294	0.5294	0.5294	0.9412
Precision ₁	0.9643	1.0000	1.0000	1.0000
Detection efficiency (s)	0.8923	2.8921	4.8921	7.0131
FNR ₁	0.4600	0.4706	0.4706	0.0588
F1-Score ₁	0.7071	0.6825	0.6825	0.9700

The detection results for python packages show that Syft, OpenSCA, and Dependency-check have the same detection accuracy, 52.94%, 52.94%, 52.94%, the reason is that the three tools get the component information by detecting the configuration files such as requirements.txt, Pipfile.lock and so on. But not only the contents of the configuration file may be lagged or incorrect, but also the import of packages in Python is not limited to the configuration file, and there are many cases where the import of packages can be done without a configuration file: 1) one can import dynamically through importlib. 2) one can import through the CLI [7].3) Conditional import. 4) Setting environment variables and so on.

Many ways. Comchecker not only analyzes the configuration file , but also analyzes the component source code , the use of components even if not declared through the configuration file , will be analyzed by our tool .

In the detection results in Table 3, the three tools have similar detection accuracy, and all of them identify the components in requirements.txt, but do not validate the content, in which unavailable components and components whose sources cannot be reached are not recognized. When declaring the component version number, in order to better meet the requirements of program operation, it is often the case that a certain version number is not specified, but a range of versions to meet the requirements of program operation is given. In our test data set, of the 82 components referenced by requirements.txt, only 54 explicitly specify a specific version number, and the remaining 28 component version numbers are declared by specifying a range, such as "Cython>=3.0", Syft, OpenSCA, and Dependency-check three tools when encountering similar situations, the results of the detection are the range of the version number, and can not determine the specific version number of the component used, the above reasons lead to the fact that they do not have the ability to detect component confusion.

Two non-component results were detected in Syft's detection results, which we suspect is due to other detection logic in the tool, which affects the accuracy under the test data set detection.

Table 4

Python dependency confusion identification results

Tool	Syft	OpenSCA	Dependency-check	Comchecker
CFC	0	0	0	13
CFI	0	0	18	18
accuracy ₂	/	/	0.7000	0.9600
precision ₂	/	/	1.0000	1.0000
Detection efficiency (s)	/	/	6.9721	9.1423
FNR ₂	/	/	0.4545	0.0606
F1-Score ₂	/	/	0.7143	0.9394

There are 18 components in the confusion test data set after component information or source code is obfuscated, for example, requestss-1.0.0 is the confusion of requests-1.0.0. The obfuscated packages are detected by both tools, but Dependency-check does not detect code confusion, but only detects component information dependency confusion by the keywords in the code file. Comchecker not only detects component name confusion and version number confusion in configuration files, but also detects dynamically imported packages or CLI imported packages in the detection process. Our tool also shows good results in detecting obfuscated code, for example, if we add malicious code in kubepy-1.13.1/setup.py, Comchecker can detect the GruppeInstall class that we added in setup.py compared to the correct version, which can detect the execution environment and not execute it if it's not a Windows environment, the malicious code in it is not executed, thus circumventing possible sandbox analysis. Other tools cannot analyze whether obfuscated code has been added inside the component. This undoubtedly increases the risk of using unknown components. Our tool detected 13 out of 15 code-dependent confusions, and two cases of confusion were not analyzed correctly because we analyzed that the detection failed due to the fact that the target

component was not covered in our component standard database.

Since our tool pays more attention to the differences between source codes, the detection of dependency confusion outperforms the comparison tool, and the accuracy of dependency confusion detection reaches 96% , as can be seen in Table 4. To summarize, our tool performs better in dependency confusion identification as well as component name and version number identification accuracy.

The experimental results show that Comchecker demonstrates significant advantages in recognizing component information and dependency confusion, and outperforms the existing component recognition tools Syft, Dependency-check, and OpenSCA in terms of F1-Score, Accuracy, Precision, and FNR, which suggests that Comchecker is able to recognize the component information used, and can effectively cope with various confusing situations that may occur in real development scenarios. However, the detection time of Comchecker is longer than that of the comparison tools due to its more complex source code analysis mechanism. Future research will continue to optimize the performance of Comchecker to improve its detection efficiency as much as possible while ensuring accuracy.

6.limitations

This study has several limitations. First, our tool currently supports detecting components used in Java and Python, which means that our tool needs to be adapted to cope with components used in other programming languages. Second, our test data set contains a total of 154 components, and the component standard database contains 200,000 component information, but as of 2023, the number of components available for Java (Maven) is 12.2M, and the number of PyPI components is 4.8M [1], and the current information in the component standard database, which is insufficient in number, does not fully cover realistic usage scenarios, and the the test data set may not represent the use of components in real-world software. Future research needs to improve the component database more to be able to cover most of the components used in the real world. Third, we hope to improve and optimize the code tree algorithm so that it can not only ensure finer-grained code analysis and prevent the situation that components with similar codes cannot be distinguished, but also ensure the simplicity of the code tree structure and the minimization of the nodes, so as to ensure the detection efficiency. Fourth, the detection time of our tool is longer than that of the comparison tool, and we hope to improve the comparison strategy in the future to enhance the efficiency of tool detection and reduce the detection time.

7.Conclusion

We introduced tools for detecting component information and dependency confusion based on source code, bytecode. Based on the component information we have collected and a large number of comparison experiments, we have drawn some conclusions and will subsequently improve our tool by drawing conclusions. Although our tool works well in the test data set, it is still uncertain in the real world. There are still many difficulties in the future

research, and we will improve the detection ability of our tool by setting up more complex experiments in real-world environments.

Data availability

The datasets analysed during the current study are available in the repository[<https://github.com/99sao/Comchecker>]

References

- [1] Understanding Open Source Adoption: Insights from the 9th State of the Software Supply Chain Report. <https://www.sonatype.com/state-of-the-software-supply-chain> (2023).
- [2] Vulnerable and outdated components - OWASP Top 10. <https://owasp.org/www-project-top-ten/> (2021).
- [3] Ohm, M., Plate, H., Sykosch, A. & Meier, M. Backstabber's Knife Collection: A review of open source software supply chain attacks. *Lecture Notes in Computer Science* 23 – 32 (2020) doi:10.1007/978-3-030-52683-2_2.
- [4] Qianxin:Announcement-of-ccleaner-malware. <https://ti.qianxin.com/blog/articles/announcement-of-ccleaner-malware/> (2015).
- [5] Birsan, A. Dependency confusion: how I hacked into Apple, Microsoft and dozens of other companies. *Medium* (2023).
- [6] Team, P. R. Typosquatting campaign targets Python developers. *Phylum* <https://blog.phylum.io/typosquatting-campaign-targets-python-developers/> (2024).
- [7] Sangwan, S. Dependency confusion attack – What, why, and how? *RedHunt Labs* <https://redhuntlabs.com/blog/dependency-confusion-attack-what-why-and-how/> (2023).
- [8] Denbraver, H. Malicious packages found to be typo-squatting in Python Package Index. *Snyk* <https://snyk.io/blog/malicious-packages-found-to-be-typo-squatting-in-pypi/> (2019).
- [9] Dunn, J. E. PyPI Python repository hit by typosquatting sneak attack. *Sophos News* <https://news.sophos.com/en-us/2017/09/19/pypi-python-repository-hit-by-typosquatting-sneak-attack/> (2017).
- [10] Spring, T. Attackers Use Typo-Squatting To Steal npm Credentials. *Threatpost* (2017).
- [11] Imtiaz, N., Khanom, A. & Williams, L. Open or sneaky? fast or slow? light or heavy?: Investigating security releases of open source packages. *IEEE Transactions on Software Engineering* **49**, 1542 – 1556 (2023).
- [12] Sangwan, S. Top organizations on GitHub Vulnerable to dependency Confusion attacks – Wave 3. *RedHunt Labs* <https://redhuntlabs.com/blog/top-organizations-on-github-vulnerable-to-dependency-confusion-attack/> (2023).

- [13] Zhao, Y. et al. Semantics-aware obfuscation scheme prediction for binary. *Computers & Security* **99**, 1 – 12 (2020).
- [14] snyk-labs. GitHub - Snyk-Labs/SNYNC: Mitigate Security Concerns of Dependency Confusion Supply chain Security Risks. *GitHub* <https://github.com/snyk-labs/snync>.
- [15] Sonatype-Nexus-Community. GitHub - sonatype-nexus-community/repo-diff: Compare the contents of your hosted and proxy repositories for coordinate collisions. *GitHub* <https://github.com/sonatype-nexus-community/repo-diff>.
- [16] Orca Security. Agentless Cloud Security | OrCA Security. *Orca Security* <https://orca.security/platform/agentless-sidescanning/> (2024).
- [17] Ruohonen, J. An empirical analysis of vulnerabilities in Python packages for web applications. *2018 9th International Workshop on Empirical Software Engineering in Practice (IWESEP)* 25 – 30 (2018) doi:10.1109/iwese.2018.00013.
- [18] What is SCA Security | Sonatype. <https://www.sonatype.com/resources/articles/what-is-software-composition-analysis>.
- [19] Imtiaz, N., Thorne, S. & Williams, L. A comparative study of vulnerability reporting by software composition analysis tools. *The 15th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM) (ESEM '21)* (2021) doi:10.1145/3475716.3475769.
- [20] Enck, W. & Williams, L. Top five challenges in software supply chain security: Observations from 30 industry and government organizations. *IEEE Security & Privacy* **20**, 96 – 100 (2022).
- [21] Milligan, S. Cybersecurity Venture's 2023 software supply chain attack report. *Snyk* <https://snyk.io/blog/cybersecurity-ventures-2023-software-supply-chain-attack-report/> (2023).
- [22] Team, P. R. Q1 2024 Evolution of Software Supply Chain Security Report. *Phylum* <https://blog.phylum.io/q1-2024-evolution-of-software-supply-chain-security-report/> (2024).
- [23] Braun, T. & Shitrit, L. B. Dependency confusion supply chain attacks: 49% of organizations are vulnerable. *Orca Security* <https://orca.security/resources/blog/dependency-confusion-supply-chain-attacks/> (2023).
- [24] Common Platform Enumeration (CPE) | NMAP Network Scanning. <https://nmap.org/book/output-formats-cpe.html>.
- [25] NVD - CVE-2017-16023. <https://nvd.nist.gov/vuln/detail/CVE-2017-16023>.
- [26] Porter, B., Laugstol, T. & Marbaise, K. H. Maven - Introduction to the dependency mechanism. <https://maven.apache.org/guides/introduction/introduction-to-dependency-mechanism.html>.
- [27] Rocky. GitHub - rocky/python-uncompyle6: A cross-version Python bytecode decompiler. *GitHub* <https://github.com/rocky/python-uncompyle6> (2024).
- [28] Abstract Syntax trees. *Python Documentation* <https://docs.python.org/zh-tw/3/library/ast.html> (2024).
- [29] Maven Repository. <https://mvnrepository.com/> (2024).
- [30] PYPI • The Python Package Index. *PyPI* <https://pypi.org/> (2024).
- [31] Anchore. GitHub - anchore/syft: CLI tool and library for generating a Software Bill of Materials from container images and filesystems. *GitHub* <https://github.com/a>

nchore/syft (2024).

[32] OWASP Dependency-Check | OWASP Foundation. <https://owasp.org/www-project-dependency-check/> (2024).

[33] XmirrorSecurity. GitHub - XmirrorSecurity/OpenSCA-cli. *GitHub* <https://github.com/XmirrorSecurity/OpenSCA-cli> (2024).

[34] Understanding F1 score, accuracy, ROC-AUC & PR-AUC metrics. *Deepchecks* <https://deepchecks.com/> (2024).

Acknowledgements

This work was supported by Beijing University of Posts and Telecommunications.

Author contributions

Ding Yanwen prepared the idea, software development, research, data organization, writing the original manuscript, all charts and pictures; Tian Weihang was in charge of research and review; Cui Baojiang was in charge of conceptualization, supervision, and project management. All authors reviewed the manuscript.

Ethics declarations

Competing interests

The authors declare no competing interests.